Aprendizaje Autónomo 1 — Principios de Sistemas de Software Distribuido

Nov 9, 2026 · @gabriel

Datos de la entrega

Campo	Valor
Estudiante	Gabriel Aguirre Román
Identificador (cédula)	0705081396
Materia	Principios de Sistemas de Software Distribuido
Sistema operativo	Windows 11 + WSL2 con Ubuntu
Repositorio GitHub	`uide_principios_sw_distribuido_0705081396` (público, por indicación del docente)
Enlace al repositorio	*github.com/aguirrethub/uide_principios_sw_distribuido_0705081396*
Enlace al video	*https://drive.google.com/file/d/1xQZvAU2NQmdw-Ad3-U7u3VaUzxyAOZxO/view?usp=sharing*
Segundo sistema	Máquina virtual VirtualBox 7.1.4 con Ubuntu Server 26.04 LTS

Los comandos de la Parte 1 se ejecutaron en WSL2 (Windows Subsystem for Linux), que corre un Ubuntu real sobre el kernel de Linux. La salida de `lsb_release -a` se incluye como primera captura para dejar constancia de la versión exacta.

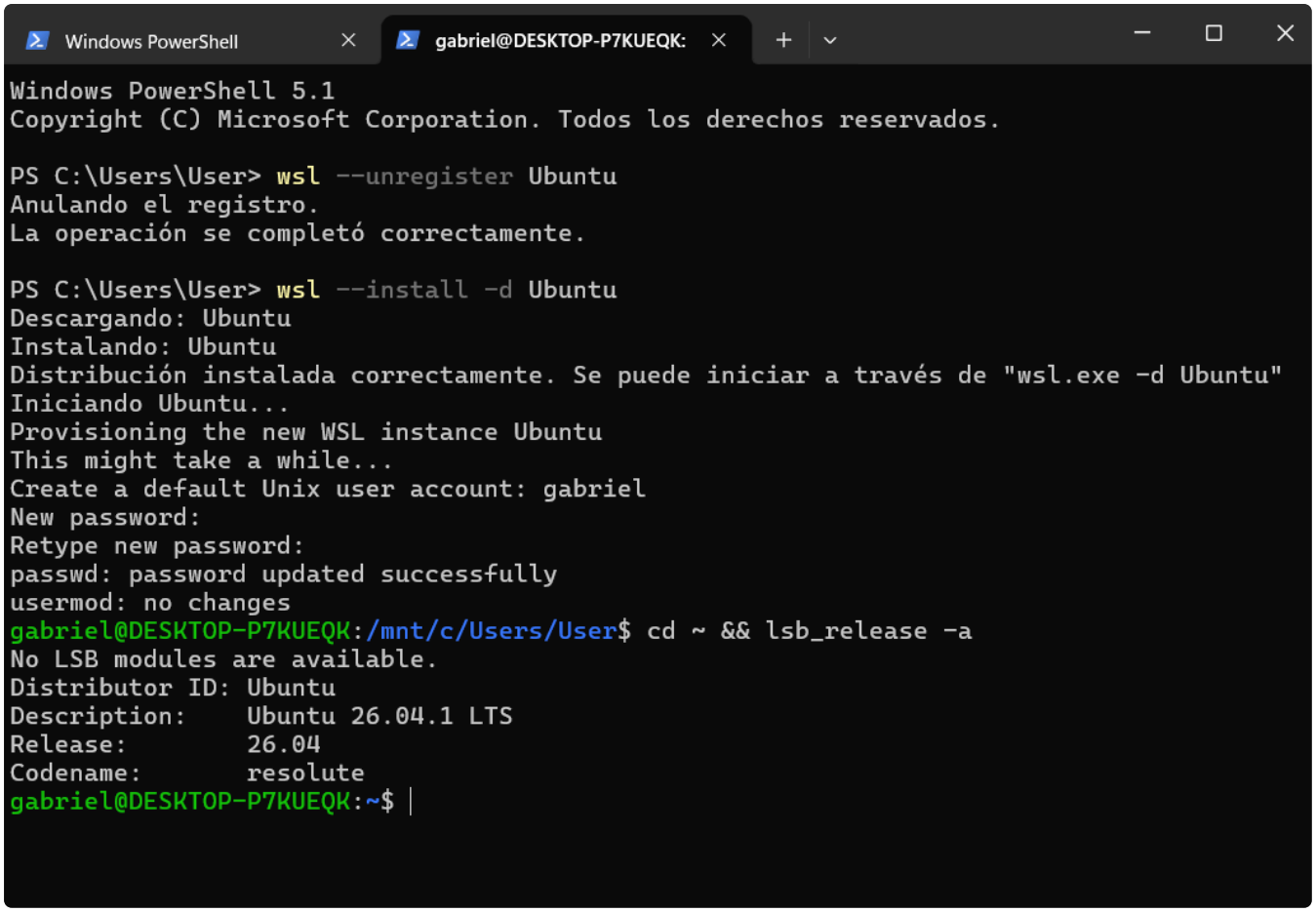


Figura 1. Instalación de WSL2 y verificación del sistema con `lsb_release -a`: Ubuntu 26.04.1 LTS, nombre en clave resolute.

Parte 1.1 — Tareas de la línea de comandos

Todos los comandos se ejecutaron en Ubuntu sobre WSL2. El archivo `logfile.txt` y una copia de `marks.csv` se colocaron al mismo nivel que `cli_assignment`, tal como pide el enunciado.

Preparación previa (no cuenta como tarea; genera el `logfile.txt` de 1000 líneas que exigen las tareas 14 a 17):

```
seq 1 1000 | awk '{print strftime("%Y-%m-%d %H:%M:%S"), "[INFO] host-1 :
evento de prueba (id=" $1 ")"}' > logfile.txt
```

Los 26 comandos

```
Sistema: Ubuntu sobre WSL2 (Windows 11)

1:  mkdir cli_assignment
2:  cd cli_assignment
3:  touch stuff.txt
```

```
4:  cat > stuff.txt
       (se escriben varias líneas y se cierra con Ctrl+D)
5:  wc -lw stuff.txt
6:  cat >> stuff.txt
       (se agregan más líneas y se cierra con Ctrl+D)
7:  mkdir draft
8:  mv stuff.txt draft/
9:  cd draft && touch .secret.txt
10: cd .. && cp -r draft final
11: mv draft draft.remove
12: mv draft.remove final/
13: ls -laR
14: head -n 100 ../logfile.txt
15: head -n 100 ../logfile.txt > logs_top_100.txt
16: tail -n 100 ../logfile.txt
17: tail -n 100 ../logfile.txt > logs_bottom_100.txt
18: cat logs_top_100.txt logs_bottom_100.txt > logs_snapshot.txt
19: echo "0705081396: Esta es una gran tarea" >> logs_snapshot.txt && date >>
logs_snapshot.txt
20: less ../logfile.txt
       (se sale con la tecla q)
21: cut -d'%' -f1 marks.csv | tail -n +2 > student_names.txt
22: cut -d'%' -f4 marks.csv | tail -n +2 | sort -n
23: awk -F'%' 'NR>1 {sum+=$3; n++} END {printf "Promedio subject_2: %.2f\n",
sum/n}' marks.csv
24: awk -F'%' 'NR>1 {sum+=$3; n++} END {printf "Promedio subject_2: %.2f\n",
sum/n}' marks.csv > done.txt
25: mv done.txt final/
26: mv final/done.txt final/average.txt
```

Qué hace cada comando y qué devuelve

#	Comando	Qué hace	Resultado verificado
1	`mkdir`	Crea el directorio de trabajo	`cli_assignment/` creado
2	`cd`	Entra al directorio	Prompt cambia de ruta
3	`touch`	Crea un archivo vacío	`stuff.txt`, 0 bytes
4	`cat >`	Redirige el teclado al archivo, sobrescribiendo	3 líneas, 16 palabras
5	`wc -lw`	Cuenta líneas y palabras en una sola pasada	`3 16 stuff.txt`

#	Comando	Qué hace	Resultado verificado
6	`cat >>`	Igual que el 4, pero `>>` agrega en lugar de sobrescribir	5 líneas, 24 palabras
7	`mkdir draft`	Crea el subdirectorio	`draft/`
8	`mv`	Mueve el archivo al subdirectorio	`draft/stuff.txt`
9	`touch .secret.txt`	El punto inicial lo vuelve oculto	Solo visible con `ls -a`
10	`cp -r`	Copia recursiva; `-r` es obligatorio para directorios	`final/` con el mismo contenido
11	`mv draft draft.remove`	Con un destino que no existe, `mv` renombra	`draft.remove/`
12	`mv draft.remove final/`	Con un destino que sí existe, `mv` mueve dentro	`final/draft.remove/`
13	`ls -laR`	Lista recursivamente el árbol completo, incluidos los ocultos, con sus permisos	Columna `drwxr-xr-x` / `-rw-r--r--`
14	`head -n 100`	Primeras 100 líneas	100 líneas en pantalla
15	`head … > archivo`	Lo mismo, redirigido a archivo	`logs_top_100.txt`, 100 líneas
16	`tail -n 100`	Últimas 100 líneas	100 líneas en pantalla
17	`tail … > archivo`	Lo mismo, redirigido	`logs_bottom_100.txt`, 100 líneas
18	`cat a b > c`	Concatena en orden	`logs_snapshot.txt`, 200 líneas
19	`echo >>` y `date >>`	Agregan identificador y fecha al final	202 líneas
20	`less`	Visor paginado; no modifica nada	Se sale con `q`
21	`cut -d'%' -f1`	Extrae la columna 1; `tail -n +2` quita el encabezado	1000 nombres
22	`cut … | sort -n`	Extrae `subject_3` y ordena numéricamente	De 30.16 a 99.98

#	Comando	Qué hace	Resultado verificado
23	`awk -F'%'`	Suma la columna 3 y divide por el total	**64.72**
24	`awk … > done.txt`	Guarda el promedio en archivo	`done.txt`
25	`mv done.txt final/`	Mueve a `final`	`final/done.txt`
26	`mv … average.txt`	Renombra dentro del mismo directorio	`final/average.txt`

Dos observaciones sobre el enunciado

La tarea 15 tiene un error. Pide crear `logs_top_100.txt` con las *últimas* 100 líneas, pero el nombre del archivo y la tarea 14 que lo precede indican que deben ser las *primeras*. Se resolvió con `head`, que es lo coherente con el resto de la secuencia.

Por qué `tail -n +2` en las tareas 21 y 22. El enunciado pide la columna sin encabezado. `cut` por sí solo devuelve también la fila `student_name`, así que se encadena `tail -n +2` para empezar desde la segunda línea.

Estructura final del directorio

```
logfile.txt
marks.csv
cli_assignment/
├── marks.csv
├── logs_top_100.txt
├── logs_bottom_100.txt
├── logs_snapshot.txt
├── student_names.txt
└── final/
    ├── .secret.txt
    ├── stuff.txt
    ├── average.txt
    └── draft.remove/
        ├── .secret.txt
        └── stuff.txt
```

Evidencias de ejecución

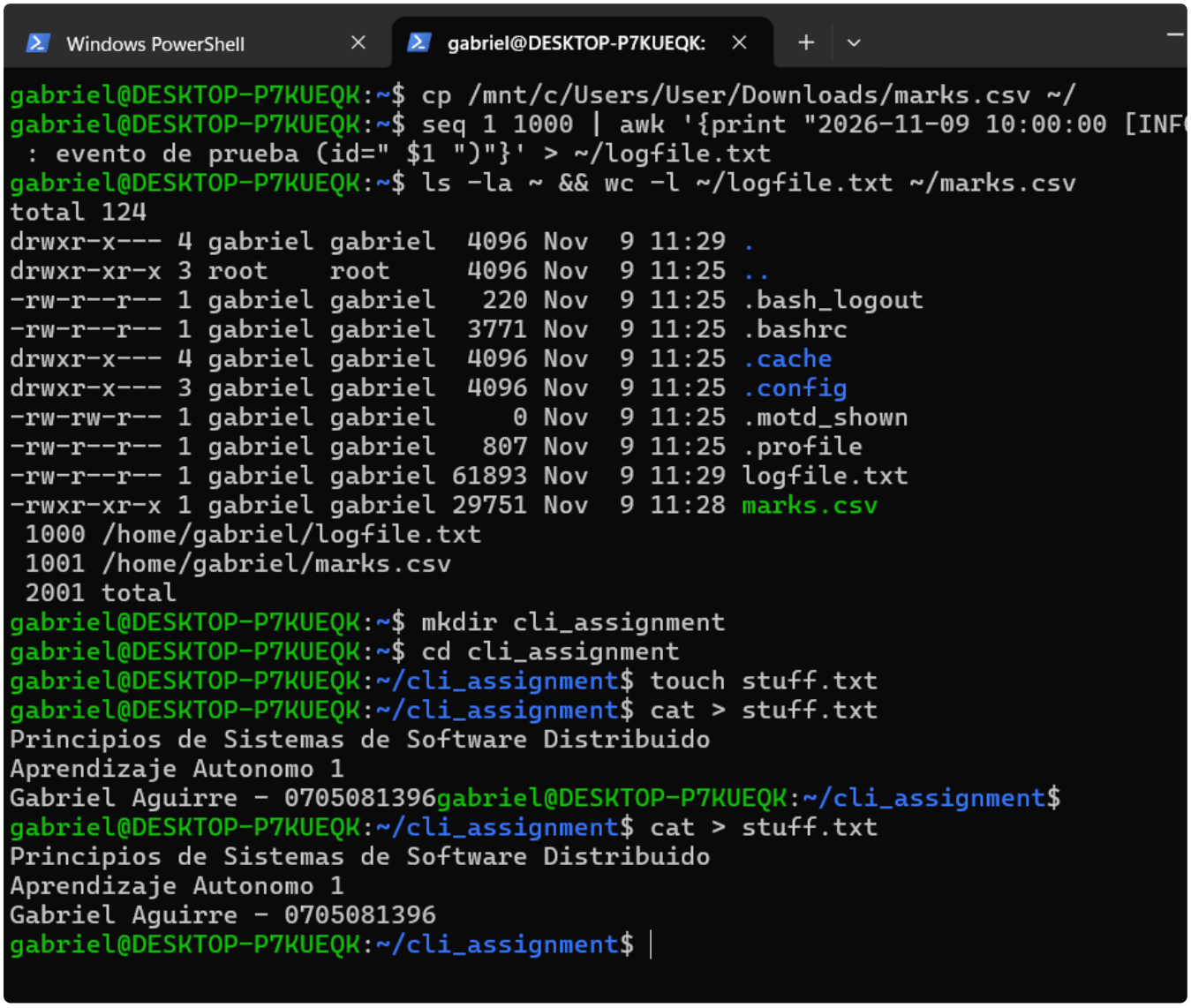


Figura 2. Preparación del entorno: copia de `marks.csv` desde Windows al home de Ubuntu y generación de `logfile.txt` con 1000 líneas.

```
gabriel@DESKTOP-P7KUEQK:~$ mkdir cli_assignment
gabriel@DESKTOP-P7KUEQK:~$ cd cli_assignment
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ touch stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat > stuff.txt
Principios de Sistemas de Software Distribuido
Aprendizaje Autonomo 1
Gabriel Aguirre - 0705081396gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat > stuff.txt
Principios de Sistemas de Software Distribuido
Aprendizaje Autonomo 1
Gabriel Aguirre - 0705081396
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ wc -lw stuff.txt
 3 13 stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat >> stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
```

Figura 3. Tareas 1 a 5. El conteo devuelve 3 líneas y 13 palabras.

```
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat >> stuff.txt
hola la lista de tareas esta en proceso

gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ wc -lw stuff.txt
  5  21 stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mkdir draft
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mv stuff.txt draft/
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cd draft && touch .secret.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment/draft$ ls -a
.  ..  .secret.txt  stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment/draft$
```

Figura 4. Tareas 6 a 9. Tras añadir texto el archivo pasa a 5 líneas y 21 palabras; `ls -a` revela el archivo oculto `.secret.txt`.

```
gabriel@DESKTOP-P7KUEQK:~/cli_assignment/draft$ cd ..
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cp -r draft final
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mv draft draft.remove
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mv draft.remove final/
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ ls -la
total 12
drwxr-xr-x 3 gabriel gabriel 4096 Nov  9 11:37 .
drwxr-x--- 5 gabriel gabriel 4096 Nov  9 11:29 ..
drwxr-xr-x 3 gabriel gabriel 4096 Nov  9 11:37 final
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
```

Figura 5. Tareas 10 a 13. Copia de `draft` a `final`, renombrado a `draft.remove` y traslado dentro de `final`; el listado con permisos confirma el resultado.

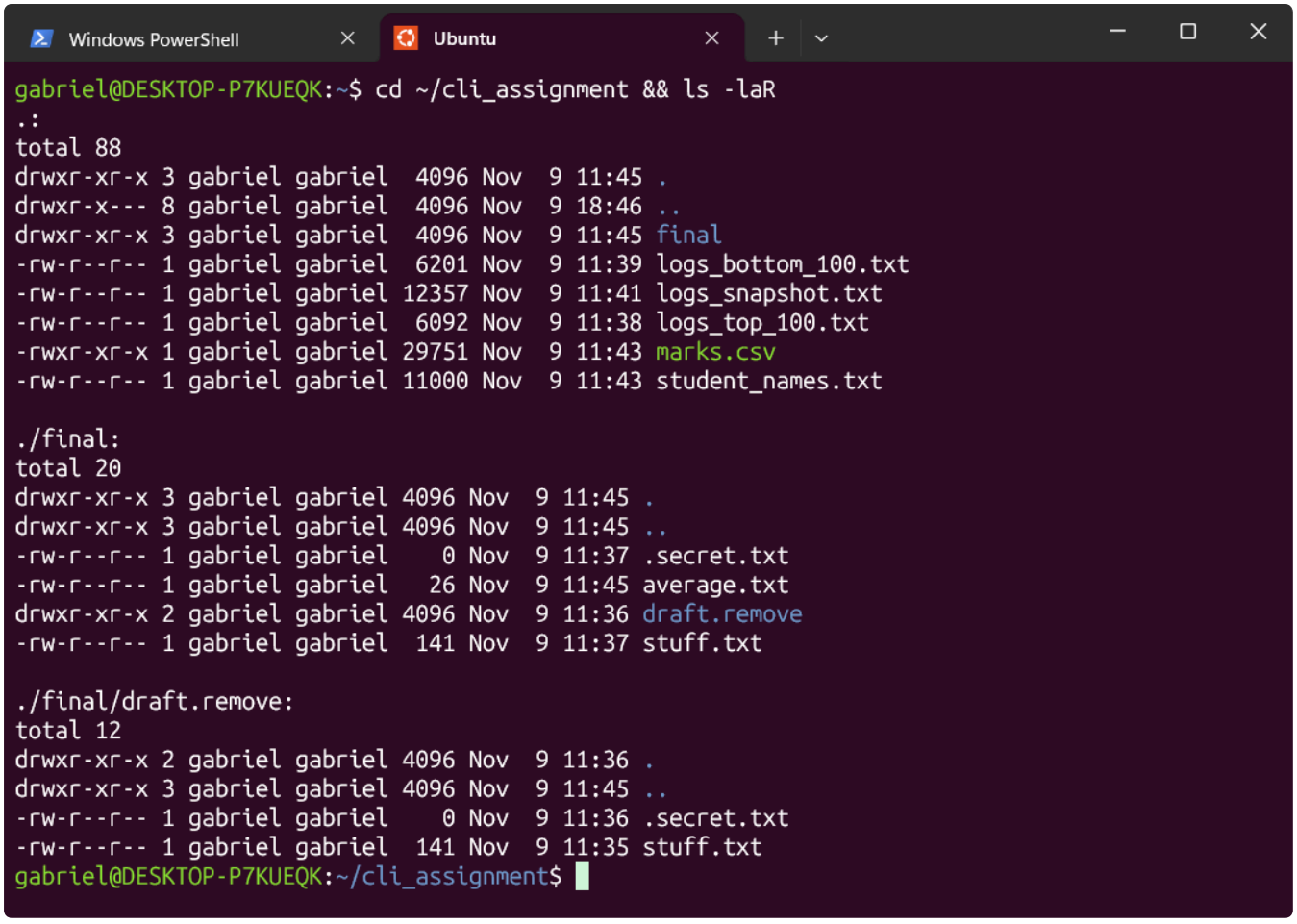


Figura 6. Tarea 13. `ls -laR` recorre el árbol entero desde `cli_assignment`: el directorio raíz, `final` y `final/draft.remove`, cada uno con sus permisos. La opción `-R` es la que añade los subdirectorios, que `ls -la` por sí solo no muestra.

```
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=100)
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ head -n 100 ../logfile.txt > logs_top_100.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ tail -n 100 ../logfile.txt
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=901)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=902)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=903)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=904)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=905)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=906)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=907)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=908)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=909)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=910)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=911)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=912)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=913)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=914)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=915)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=916)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=917)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=918)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=919)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=920)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=921)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=922)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=923)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=924)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=925)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=926)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=927)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=928)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=929)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=930)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=931)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=932)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=933)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=934)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=935)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=936)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=937)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=938)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=939)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=940)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=941)
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=942)
```

Figura 7. Tareas 14 y 15. Lectura de las primeras 100 líneas y redirección a `logs_top_100.txt`.

```
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ tail -n 100 ../logfile.txt > logs_bottom_100.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ wc -l logs_top_100.txt logs_bottom_100.txt
  100 logs_top_100.txt
  100 logs_bottom_100.txt
  200 total
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat logs_top_100.txt logs_bottom_100.txt > logs_sna
pshot.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ echo "0705081396: Esta es una gran tarea" >> logs_s
napshot.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ date >> logs_snapshot.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ tail -n 3 logs_snapshot.txt && wc -l logs_snapshot.
txt
2026-11-09 10:00:00 [INFO] host-1 : evento de prueba (id=1000)
0705081396: Esta es una gran tarea
Mon Nov  9 11:41:08 -05 2026
202 logs_snapshot.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
```

Figura 8. Tareas 16 a 19. Los dos archivos parciales tienen 100 líneas cada uno; tras concatenar y anexar el identificador y la fecha, `logs_snapshot.txt` queda con 202 líneas.

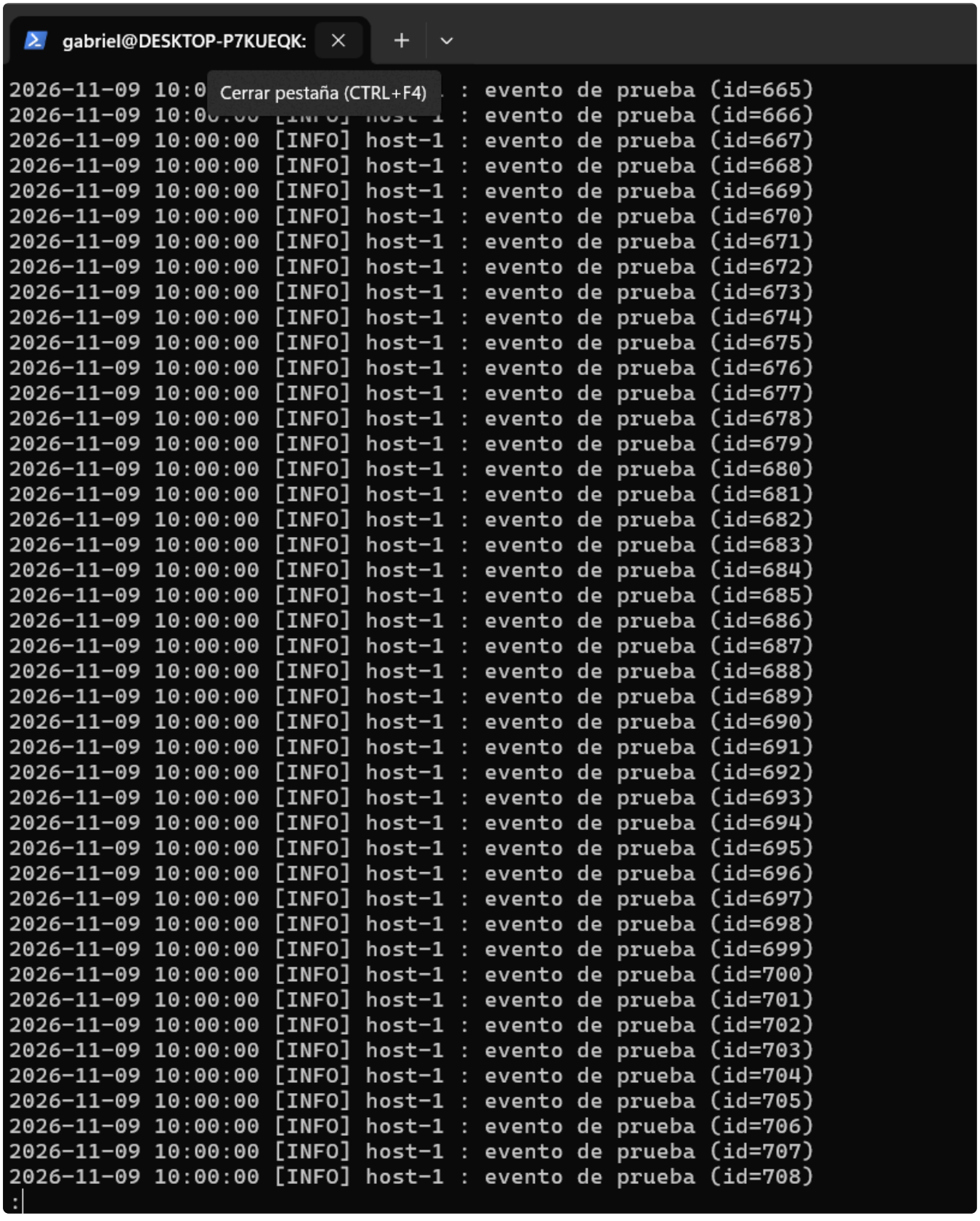


Figura 9. Tarea 20. El archivo abierto con `less`, que pagina el contenido sin cargarlo entero en memoria.

```
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ less ../logfile.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cp ~/marks.csv .
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cut -d'%' -f1 marks.csv | tail -n +2 > student_name
s.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ head -5 student_names.txt && wc -l student_names.tx
t
h4B24hCpiU
hO6HIo2wgL
esRsGtZvar
KVUDiaiKzD
SZbdRyoxNP
1000 student_names.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
```

Figura 10. Tarea 21. `cut` extrae la primera columna y `tail -n +2` descarta el encabezado: 1000 nombres.

```
gabriel@DESKTOP-P7KUEQK:
97.65
97.69
97.71
97.79
97.86
97.99
98.0
98.0
98.05
98.21
98.22
98.24
98.37
98.46
98.47
98.57
98.8
98.8
98.87
98.89
98.94
99.02
99.02
99.06
99.1
99.23
99.28
99.42
99.42
99.49
99.49
99.69
99.85
99.88
99.92
99.95
99.98
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cut -d'%' -f4 marks.csv | tail -n +2 | sort -n |
ad -5
30.16
30.23
30.35
30.66
30.67
```

Figura 11. Tarea 22. Las calificaciones de `subject_3` ordenadas numéricamente, de 30.16 a 99.98.

```
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ awk -F'%' 'NR>1 {sum+=$3; n++} END {printf "Promedi
o subject_2: %.2f\n", sum/n}' marks.csv
Promedio subject_2: 64.72
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ awk -F'%' 'NR>1 {sum+=$3; n++} END {printf "Promedi
o subject_2: %.2f\n", sum/n}' marks.csv > done.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mv done.txt final/
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ mv final/done.txt final/average.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$ cat final/average.txt && ls -la final
Promedio subject_2: 64.72
total 20
drwxr-xr-x 3 gabriel gabriel 4096 Nov  9 11:45 .
drwxr-xr-x 3 gabriel gabriel 4096 Nov  9 11:45 ..
-rw-r--r-- 1 gabriel gabriel    0 Nov  9 11:37 .secret.txt
-rw-r--r-- 1 gabriel gabriel   26 Nov  9 11:45 average.txt
drwxr-xr-x 2 gabriel gabriel 4096 Nov  9 11:36 draft.remove
-rw-r--r-- 1 gabriel gabriel  141 Nov  9 11:37 stuff.txt
gabriel@DESKTOP-P7KUEQK:~/cli_assignment$
```

Figura 12. Tareas 23 a 26. El promedio de `subject_2` es 64.72; el archivo se guarda, se mueve a `final` y se renombra a `average.txt`.

Parte 2.1 — Repositorio de GitHub

El repositorio se creó bajo la cuenta `aguirrethub`, con el nombre que pide el enunciado sustituyendo `ci` por la cédula.

Enlace: https://github.com/aguirrethub/uide_principios_sw_distribuido_0705081396

Requisito	Estado
Nombre `uide_principios_sw_distribuido_0705081396`	Cumplido
Visibilidad pública (el enunciado pedía privada; el docente indicó dejarlo público)	Cumplido
Carpeta `Autonomo1`	Creada con `README.md`
Carpeta `Autonomo2`	Creada con `README.md`
Invitación al instructor	No aplica: al ser público, basta con compartir el enlace

Git no versiona directorios vacíos, solo archivos. Por eso cada carpeta se creó con un `README.md` dentro: al escribir la ruta `Autonomo1/README.md` en el formulario de GitHub, la carpeta se genera junto con el archivo en un solo commit.

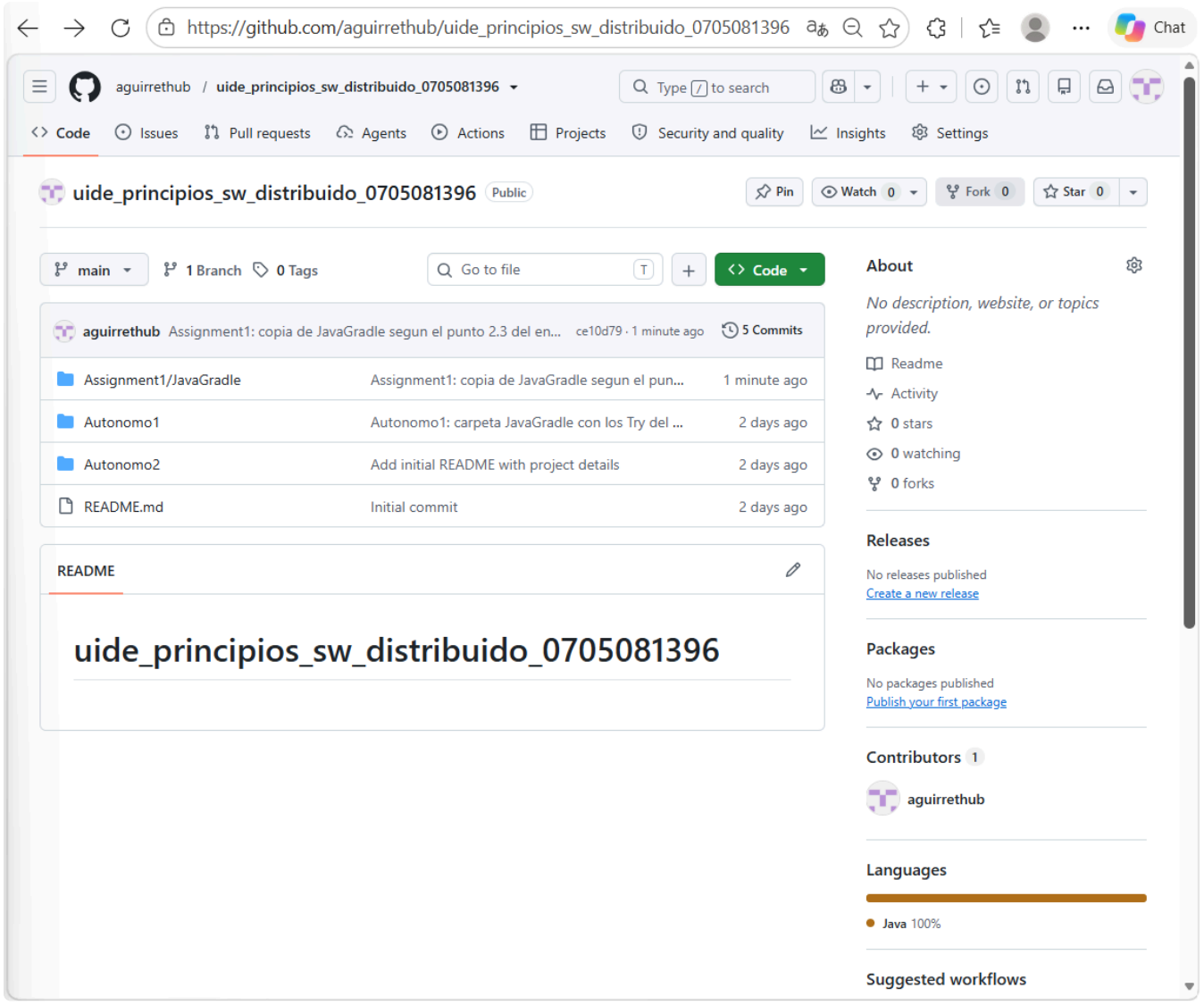


Figura 13. Página principal del repositorio. La etiqueta Public confirma la visibilidad solicitada por el docente. Se ven las tres carpetas: `Autonomo1` y `Autonomo2`, que pide el punto 2.1, y `Assignment1/JavaGradle`, que pide el punto 2.3.

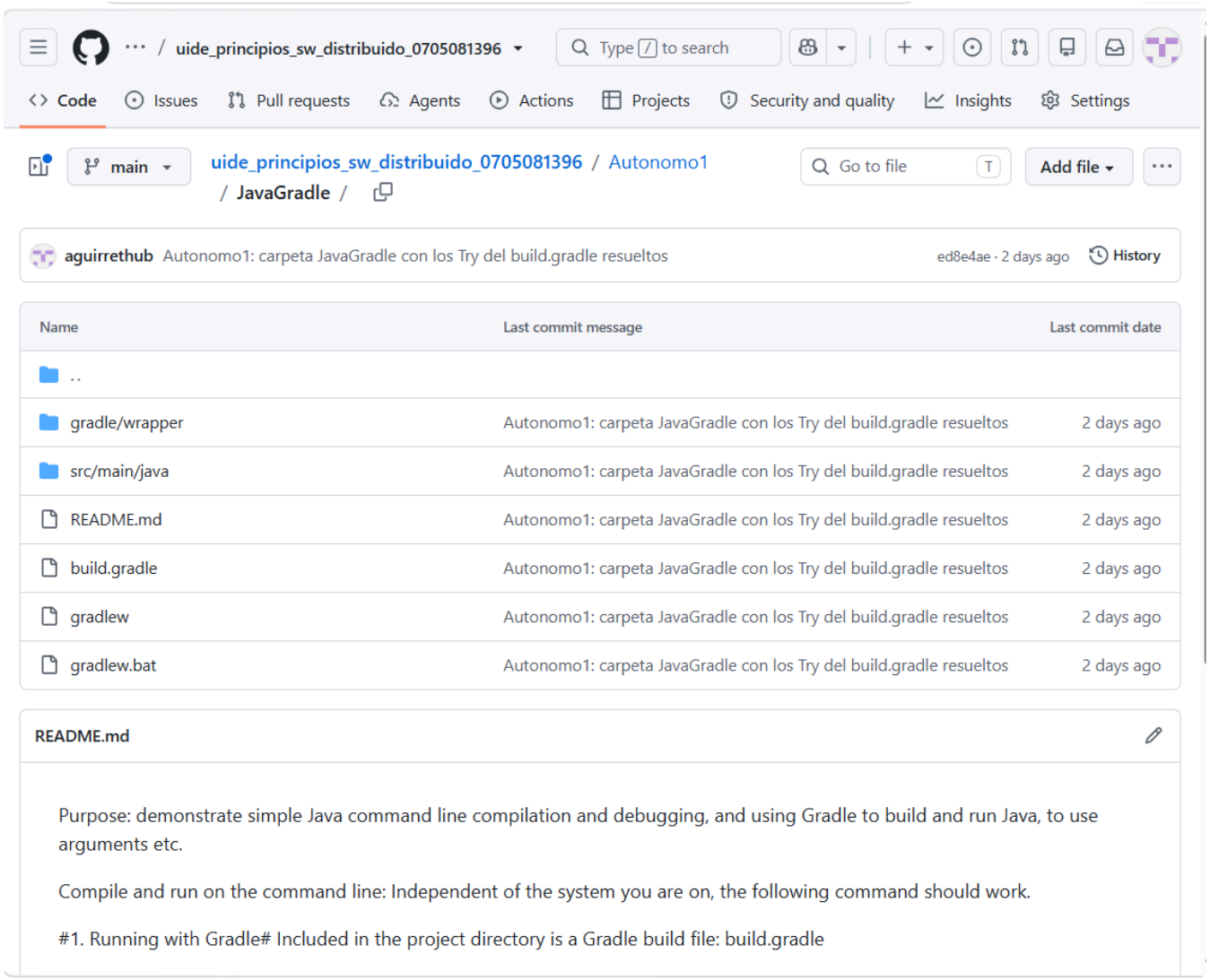


Figura 14. Contenido de `Autonomo1/JavaGradle` en el repositorio: el `build.gradle` modificado, las fuentes en `src/main/java` y el wrapper de Gradle.

Parte 2.2 — Ejecución de Network/SimpleGrabHttpURL

El ejemplo se ejecutó con `gradle run` y terminó en `BUILD SUCCESSFUL`, mostrando primero las cabeceras HTTP de la respuesta y después el HTML completo de la página.

Preparación del entorno

La versión de Gradle que trae Ubuntu 26.04 por defecto es la 4.4.1, de 2017, y no es compatible con Java 17. Tampoco servía el wrapper incluido en el repositorio, que apunta a Gradle 6.6.1 (el soporte de Java 17 llegó en Gradle 7.3). Se instaló entonces Gradle 8.14.3 de forma manual.

Se eligió la rama 8 y no la 9 porque Gradle 9 eliminó la propiedad `mainClassName`, que es la que usan los `build.gradle` de este repositorio. Con la 8.14.3 los archivos funcionan sin modificarlos; solo aparece un aviso de función obsoleta, que no es un error.

```
sudo apt install -y openjdk-17-jdk git unzip wget
wget -q https://services.gradle.org/distributions/gradle-8.14.3-bin.zip -P
/tmp
sudo unzip -q -d /opt/gradle /tmp/gradle-8.14.3-bin.zip
echo 'export PATH=/opt/gradle/gradle-8.14.3/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Ejecución

```
cd ~/ser321examples/Network/SimpleGrabHttpURL
gradle run
```

Para ver solo las cabeceras, sin el HTML:

```
gradle run -q --console=plain 2>/dev/null | head -20
```

Cabeceras devueltas por el servidor en la ejecución real: `HTTP/1.1 200 OK`, `Server: cloudflare`, `Content-Type: text/html; charset=utf-8`, `Cache-Control: public, max-age=10800`.

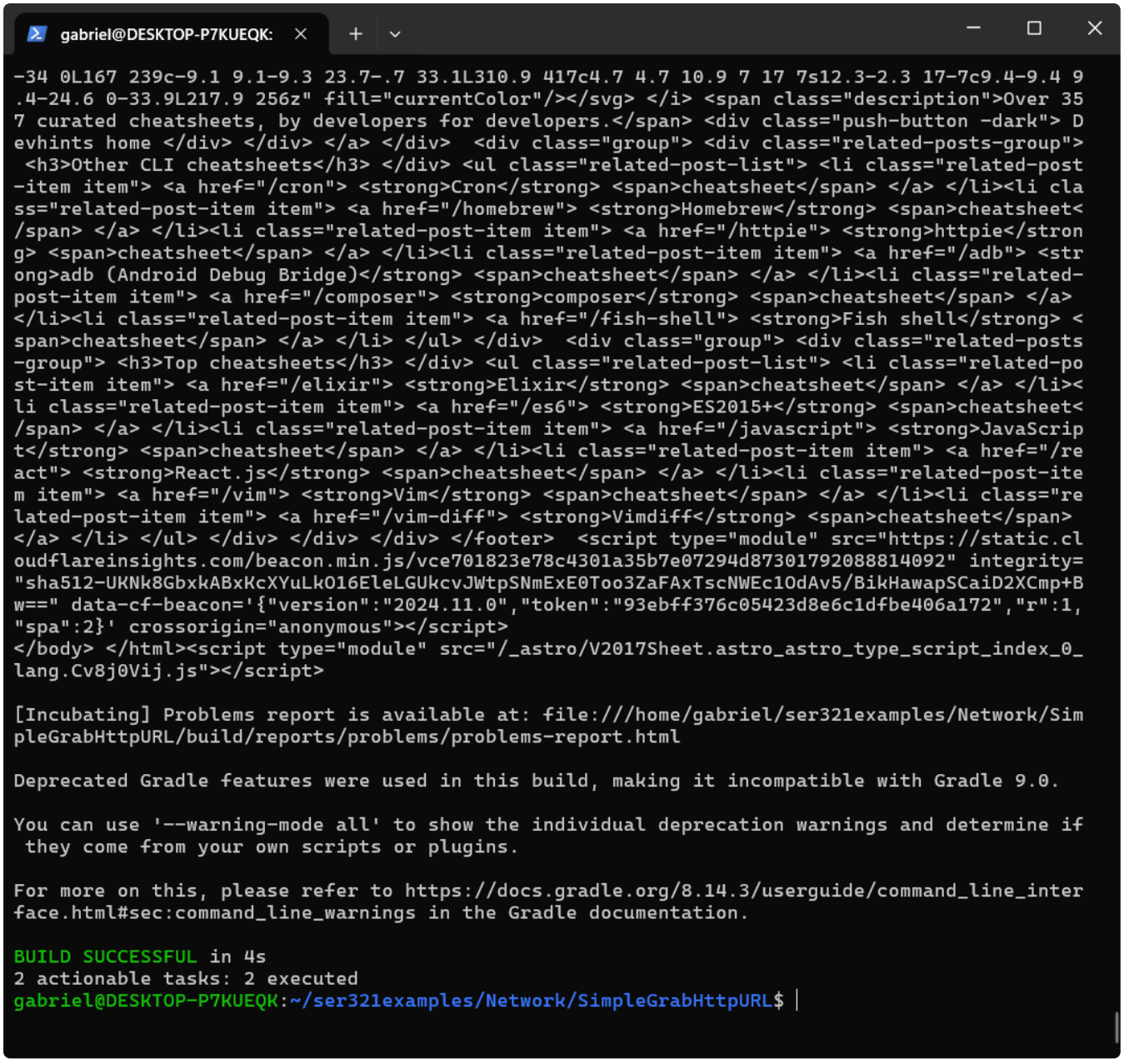


Figura 15. Ejecución de `gradle run` sobre SimpleGrabHttpURL. El programa imprime el HTML de la página y la compilación termina en `BUILD SUCCESSFUL`. El aviso sobre funciones obsoletas corresponde a `mainClassName` y no es un error.

```
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Network/SimpleGrabHttpURL$ gradle run -q --console=p
lain 2>/dev/null | head -20
Key : Transfer-Encoding ,Value : [chunked]
Key : null ,Value : [HTTP/1.1 200 OK]
Key : CF-RAY ,Value : [a57e738138d79afb-GYE]
Key : Server ,Value : [cloudflare]
Key : vary ,Value : [accept-encoding]
Key : Access-Control-Allow-Origin ,Value : [*]
Key : Server-Timing ,Value : [cfEdge;dur=18,cfOrigin;dur=0, cfCacheStatus;desc="HIT"]
Key : Connection ,Value : [keep-alive]
Key : cf-cache-status ,Value : [HIT]
Key : Date ,Value : [Mon, 09 Nov 2026 17:13:59 GMT]
Key : last-modified ,Value : [Mon, 09 Nov 2026 01:40:31 GMT]
Key : Strict-Transport-Security ,Value : [max-age=2592000; includeSubDomains; preload]
Key : Cache-Control ,Value : [public, max-age=10800, must-revalidate]
Key : Nel ,Value : [{"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}]
Key : x-content-type-options ,Value : [nosniff]
Key : Report-To ,Value : [{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.n
el.cloudflare.com/report/v4?s=bAnrqrA61Zu5%2BB9KTNOgYYhAQ1r4j7pTsauCDXIB2BOnaUvIuZ914obLgXiLr
tkHvPYsogPHHt3GqKM%2FvhSUaVrgef4M9A4griBeNWygOWFj7%2FtUhTw5Ese97uTXUkCMFuy6L8HoJ6HH9A%3D%3D"}
]}]
Key : referrer-policy ,Value : [strict-origin-when-cross-origin]
Key : alt-svc ,Value : [h3=":443"; ma=86400]
Key : Age ,Value : [56007]
Key : Content-Type ,Value : [text/html; charset=utf-8]
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Network/SimpleGrabHttpURL$
```

Figura 16. Cabeceras HTTP de la respuesta. Se distinguen `HTTP/1.1 200 OK`, el servidor `cloudflare` y el `Content-Type: text/html; charset=utf-8`, que son los elementos de las capas superiores analizados en esta parte.

Los otros dos ejemplos de la carpeta Network

Los requisitos previos piden ejecutar también `Network/SimpleGrabURL` y `Network/HTTP-JSON`. Ambos se ejecutaron con la misma instalación de Gradle 8.14.3.

`SimpleGrabURL` hace lo mismo que `SimpleGrabHttpURL` pero con la clase genérica `URLConnection` en lugar de `HttpURLConnection`. La diferencia práctica es que no expone los métodos propios de HTTP, como el código de estado de la respuesta.

```
cd ~/ser321examples/Network/SimpleGrabURL
gradle run -q --console=plain 2>/dev/null | head -25
```

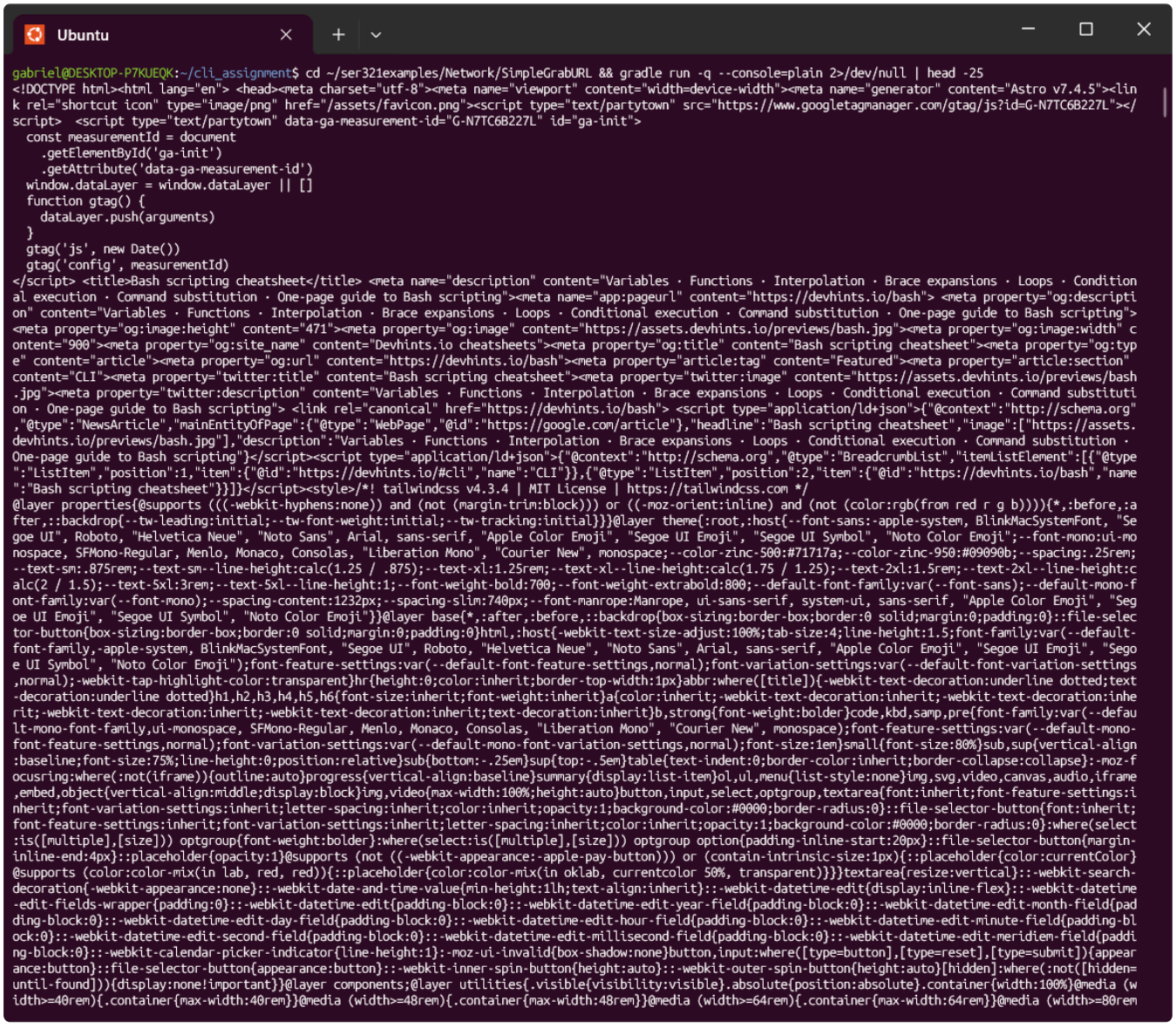


Figura 17. `SimpleGrabURL` *descarga y muestra el HTML de la misma página de prueba.*

`HTTP-JSON` va un paso más allá: consulta la API pública de GitHub y recibe la respuesta en JSON. A diferencia de los dos anteriores, este ejemplo sí declara dependencias externas en su `build.gradle` (`org.json` y `gson`), que Gradle descarga de Maven Central en la primera ejecución.

```
cd ~/ser321examples/Network/HTTP-JSON
gradle API -q --console=plain 2>/dev/null | head -25
```

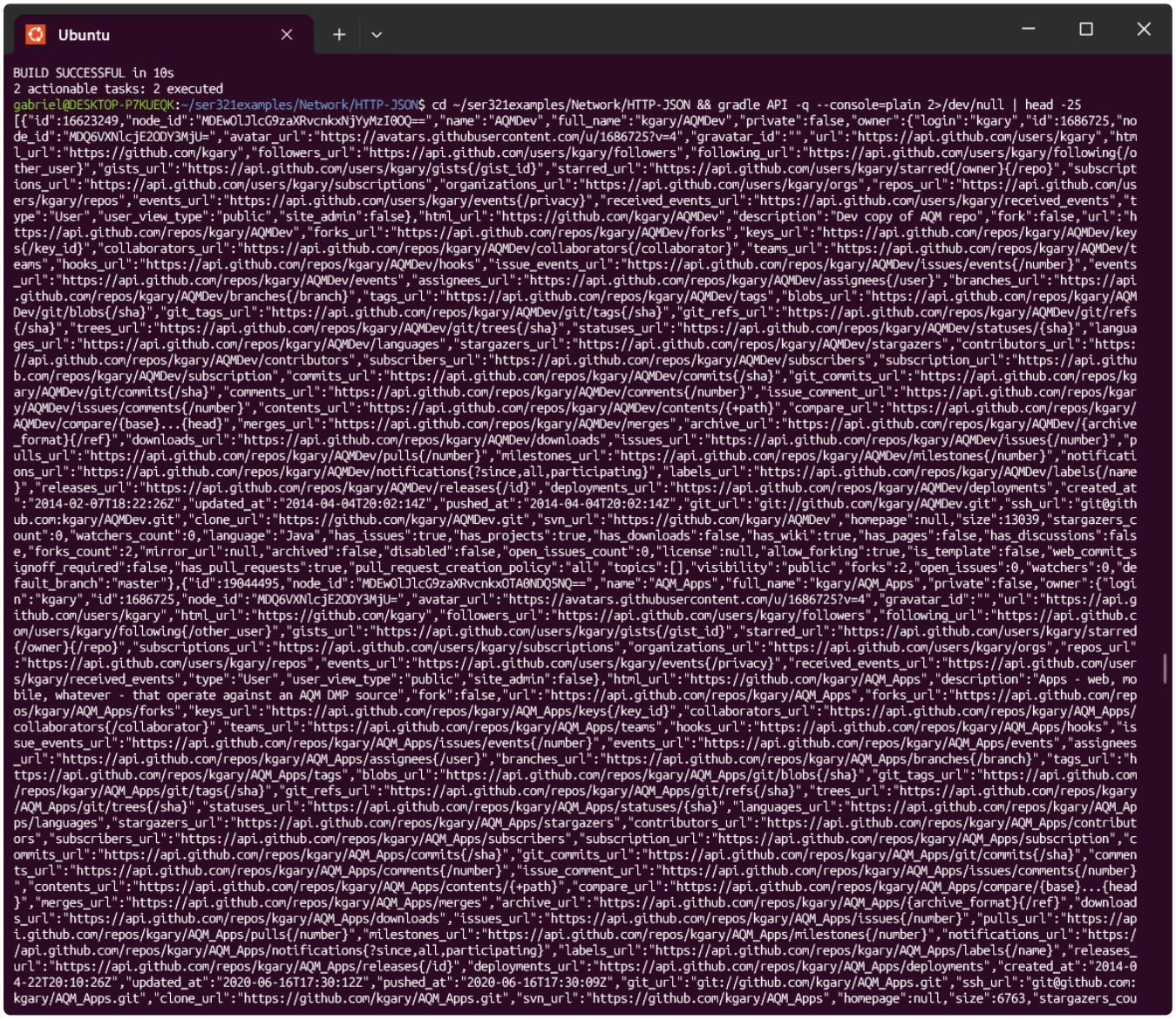


Figura 18. `HTTP-JSON` termina en `BUILD SUCCESSFUL` y devuelve el JSON con los repositorios del usuario consultado. El formato JSON corresponde a la capa 6 del modelo OSI: es la representación acordada para que ambos extremos interpreten los mismos datos.

El archivo build.gradle

```
apply plugin: 'application'

application {
    mainClassName = 'SimpleGrabHttpURL'
    description = "SimpleGrabHttpURL Example"
}

run {
  args 'https://devhints.io/bash'
}
```

El plugin `application` aporta la tarea `run`. `mainClassName` indica qué clase contiene el `main`. El bloque `run` fija la URL por defecto, de modo que `gradle run` funciona sin pasar argumentos. Para consultar otra dirección se usa `gradle run --args 'https://example.com'`.

Qué hace cada parte del programa

Línea o bloque	Función
`import java.net.*`	Trae `URL` y `HttpURLConnection`, las clases que manejan la conexión HTTP
`import java.io.*`	Trae `BufferedReader` e `InputStreamReader` para leer la respuesta
`if (args.length != 1)`	Valida que se reciba exactamente una URL; si no, avisa y termina con `System.exit(0)`
`HttpURLConnection conn = null`	Declara la conexión fuera del `try` para poder cerrarla en el `finally`
`new URL(args[0])`	Convierte el texto en un objeto URL, separando protocolo, host y ruta
`.openConnection()`	Crea el objeto de conexión. Todavía no envía nada por la red
`(HttpURLConnection)`	Conversión de tipo. `openConnection()` devuelve un `URLConnection` genérico; el cast da acceso a la API específica de HTTP
`conn.getHeaderFields()`	Aquí sí se dispara la petición. Devuelve un `Map` con las cabeceras de la respuesta
`for (Map.Entry...)`	Recorre ese mapa e imprime cada cabecera como clave y valor
`conn.getInputStream()`	Abre el flujo de bytes del cuerpo de la respuesta
`new InputStreamReader(...)`	Convierte esos bytes en caracteres
`new BufferedReader(...)`	Agrega un búfer para poder leer línea por línea en lugar de carácter por carácter
`while ((line = instream.readLine()) != null)`	Lee hasta que `readLine()` devuelve `null`, que marca el fin del flujo

Línea o bloque	Función
`catch (IOException exc)`	Captura fallos de red: host inexistente, conexión rechazada, tiempo agotado
`finally { instream.close(); }`	Cierra el flujo pase lo que pase, para no dejar recursos abiertos

Relación con las capas del modelo OSI

El programa opera en la **capa 7 (aplicación)**: construye una petición HTTP y lee una respuesta HTTP. No escribe sockets ni maneja puertos — de eso se encargan las capas inferiores, que `HttpURLConnection` abstrae por completo.

La cabecera `Content-Type: text/html; charset=utf-8` corresponde a la **capa 6 (presentación)**: le dice al cliente en qué formato y codificación viene el contenido, para que sepa cómo interpretar los bytes.

La cabecera `Connection: keep-alive` toca la **capa 5 (sesión)**: indica que la conexión TCP subyacente se mantiene abierta para reutilizarla en peticiones siguientes, en lugar de cerrarla tras cada intercambio.

Parte 2.3 — Entendiendo Gradle

El archivo `Gradle/JavaGradle/build.gradle` contiene un comentario `// Try:` con dos ejercicios. Ambos se resolvieron y se verificaron ejecutando la tarea con cuatro combinaciones distintas de parámetros.

Los dos ejercicios del comentario

1. Modificar `Fraction.main` para que acepte dos argumentos.
2. Crear una tarea de Gradle que reciba esos dos argumentos y funcione escribiendo `gradle runFrac -Pdenom=4 -Pnum=3` o `gradle runFrac -Pnum=3 -Pdenom=4`, y que además corra con un solo parámetro o sin ninguno.

Ejercicio 1 — Fraction.main con dos argumentos

La versión original fijaba la fracción 1/3 en el código. La modificación lee `args[0]` como numerador y `args[1]` como denominador, con tres casos:

- Dos argumentos: se usan ambos.
- Un argumento: se toma como numerador y el denominador queda en 1.
- Ningún argumento: se mantiene 1/3, el valor original.

Se agregaron dos validaciones que el enunciado no pide pero que evitan que el programa se caiga: un `catch (NumberFormatException)` si los argumentos no son enteros, y un control para rechazar el denominador cero.

```
public static void main (String args[]) {
   int num = 1;
   int denom = 3;

   try {
      if (args.length >= 2) {
         num = Integer.parseInt(args[0]);
         denom = Integer.parseInt(args[1]);
      } else if (args.length == 1) {
         num = Integer.parseInt(args[0]);
         denom = 1;
      }
   } catch (NumberFormatException e) {
      System.out.println("Los argumentos deben ser numeros enteros.");
      System.exit(1);
   }

   if (denom == 0) {
      System.out.println("El denominador no puede ser cero.");
      System.exit(1);
   }

   Fraction frac = new Fraction();
   frac.setNumerator(num);
   frac.setDenominator(denom);
   System.out.print("The fraction is: ");
   frac.print();
   System.out.println("");
}
```

Ejercicio 2 — La tarea runFrac

La clave de que el orden no importe está en `project.hasProperty()`: Gradle expone los `-P` como propiedades con nombre, no como posiciones. Por eso `-Pnum=3 -Pdenom=4` y `-Pdenom=4 -Pnum=3` producen lo mismo.

```
task runFrac(type: JavaExec) {
  group 'Fraction Tasks'
  description 'Ejecuta Fraction con -Pnum y -Pdenom en cualquier orden'

  classpath = sourceSets.main.runtimeClasspath
  main = 'Fraction'

  if (project.hasProperty("num") && project.hasProperty("denom")) {
    args(project.getProperty('num'), project.getProperty('denom'))
  } else if (project.hasProperty("num")) {
    args(project.getProperty('num'))
  } else if (project.hasProperty("denom")) {
    args(1, project.getProperty('denom'))
  }
}
```

Ejercicio adicional dentro de runAppAgain

Hay un segundo `// Try:` en la tarea `runAppAgain`, que pedía usar valores por defecto cuando no se pasa ningún parámetro. La versión original no pasaba argumentos en ese caso y `Multiply` respondía con un mensaje de error. Se agregaron dos ramas: una para cuando solo llega `num2` y otra para cuando no llega ninguno, que usa `args(1, 1)`.

Resultados verificados

Comando	Salida
`gradle runFrac -Pnum=3 -Pdenom=4`	`The fraction is: 3/4`
`gradle runFrac -Pdenom=4 -Pnum=3`	`The fraction is: 3/4`
`gradle runFrac -Pnum=5`	`The fraction is: 5/1`
`gradle runFrac`	`The fraction is: 1/3`
`gradle runAppAgain`	`1 * 1 = 1`

```
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$ cd ~/ser321examples/Gradle/JavaGr
adle
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$ cp /mnt/c/Users/User/Downloads/bu
ild.gradle .
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$ cp /mnt/c/Users/User/Downloads/Fr
action.java src/main/java/
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$ gradle runFrac -Pnum=3 -Pdenom=4
-q --console=plain
The fraction is: 3/4
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$ gradle runFrac -Pdenom=4 -Pnum=3
-q --console=plain && gradle runFrac -Pnum=5 -q --console=plain && gradle runFrac -q --consol
e=plain
The fraction is: 3/4
The fraction is: 5/1
The fraction is: 1/3
gabriel@DESKTOP-P7KUEQK:~/ser321examples/Gradle/JavaGradle$
```

Figura 19. Las cuatro variantes de `runFrac`. Los dos primeros comandos invierten el orden de los parámetros y devuelven el mismo resultado; los dos últimos comprueban el comportamiento con un solo parámetro y sin ninguno.

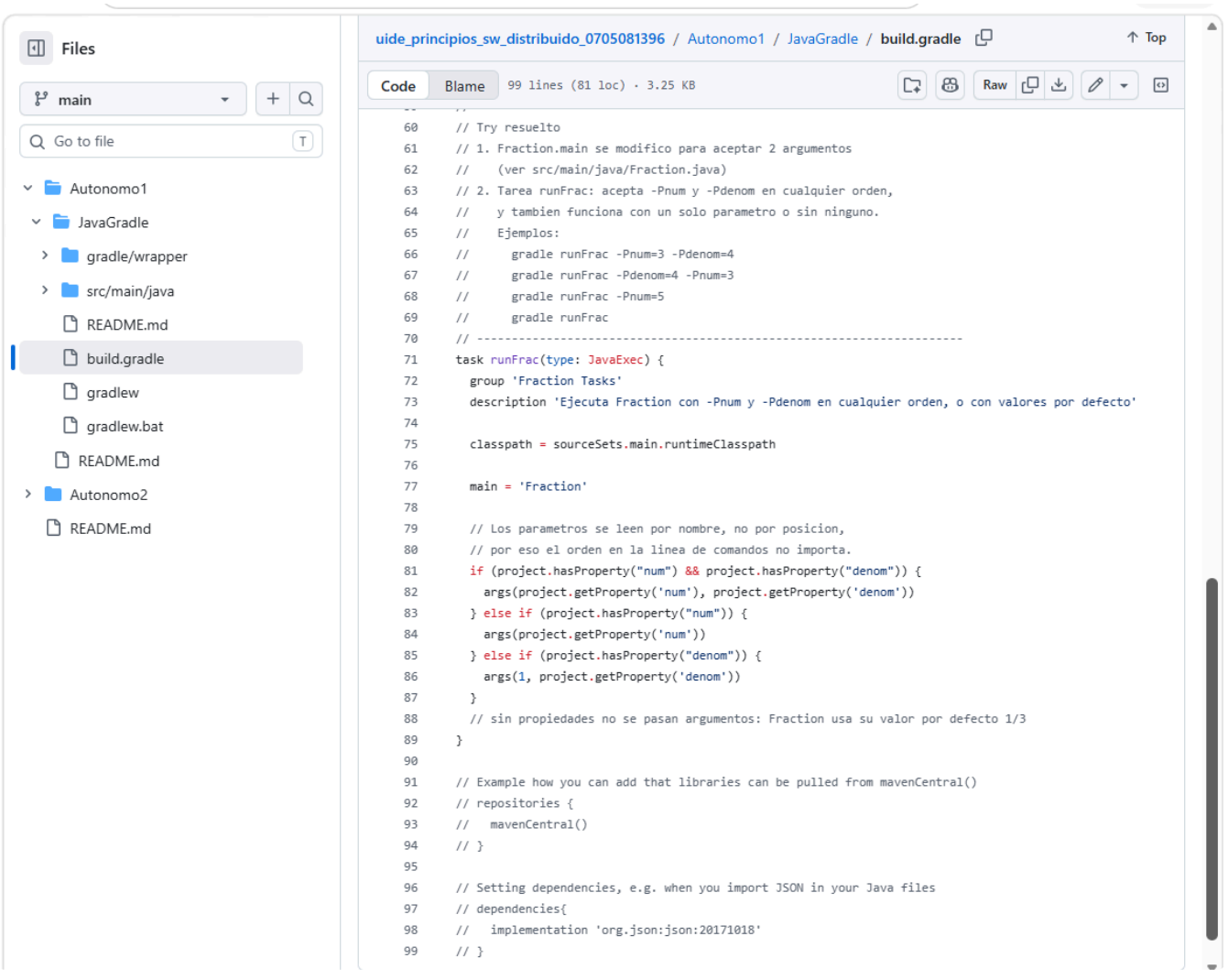


Figura 20. La tarea `runFrac` tal como quedó en el `build.gradle` del repositorio, con el comentario "Try resuelto" y la lectura de los parámetros por nombre que hace irrelevante su orden.

Una inconsistencia del enunciado

El punto 2.1 manda crear las carpetas `Autonomo1` y `Autonomo2`, pero el punto 2.3 pide copiar la carpeta Java Gradle dentro de `Assignment1`, un nombre que no aparece en ninguna otra parte del enunciado. Como ambas lecturas son defendibles, la carpeta se subió con los dos nombres: `Autonomo1/JavaGradle` y `Assignment1/JavaGradle`, con contenido idéntico. Así se cumple el punto 2.3 al pie de la letra sin dejar de respetar la estructura que define el 2.1.

Parte 2.4 — Segundo sistema y ejecución distribuida

Tipo de segundo sistema: máquina virtual. Se usó VirtualBox 7.1.4 con Ubuntu Server 26.04 LTS, nombre de host `servidor-vm`.

Por qué una máquina virtual y no AWS EC2

La intención inicial era usar una instancia EC2. Al revisar la cuenta, AWS la había cerrado el 8 de noviembre de 2026 porque el periodo gratuito de 6 meses terminó, y restaurar el acceso exigía pasar a un plan de pago. Crear otra cuenta con un correo distinto tampoco resolvía nada: la elegibilidad de la capa gratuita se evalúa por método de pago e historial, no solo por correo. El enunciado acepta explícitamente una máquina virtual como segundo sistema, así que se tomó esa vía.

Configuración de la máquina virtual

Parámetro	Valor
Hipervisor	Oracle VirtualBox 7.1.4
Sistema invitado	Ubuntu Server 26.04 LTS
Nombre de host	`servidor-vm`
Memoria	2048 MB
Procesadores	2
Disco	15 GB
Adaptador de red	**Adaptador puente** sobre Intel Wi-Fi 6 AX201
Dirección IP	192.168.100.123/24 (DHCP del router)
Java	OpenJDK 25

El **adaptador puente** es la decisión importante de esta parte. Con el modo NAT que VirtualBox trae por defecto, la máquina virtual queda detrás de la máquina anfitriona y no es alcanzable desde fuera. Con el adaptador puente, la VM pide su propia dirección al router y aparece en la red local como un equipo más. Por eso recibió `192.168.100.123`, distinta de la del anfitrión, y por eso el cliente puede conectarse a ella como a cualquier otra máquina.

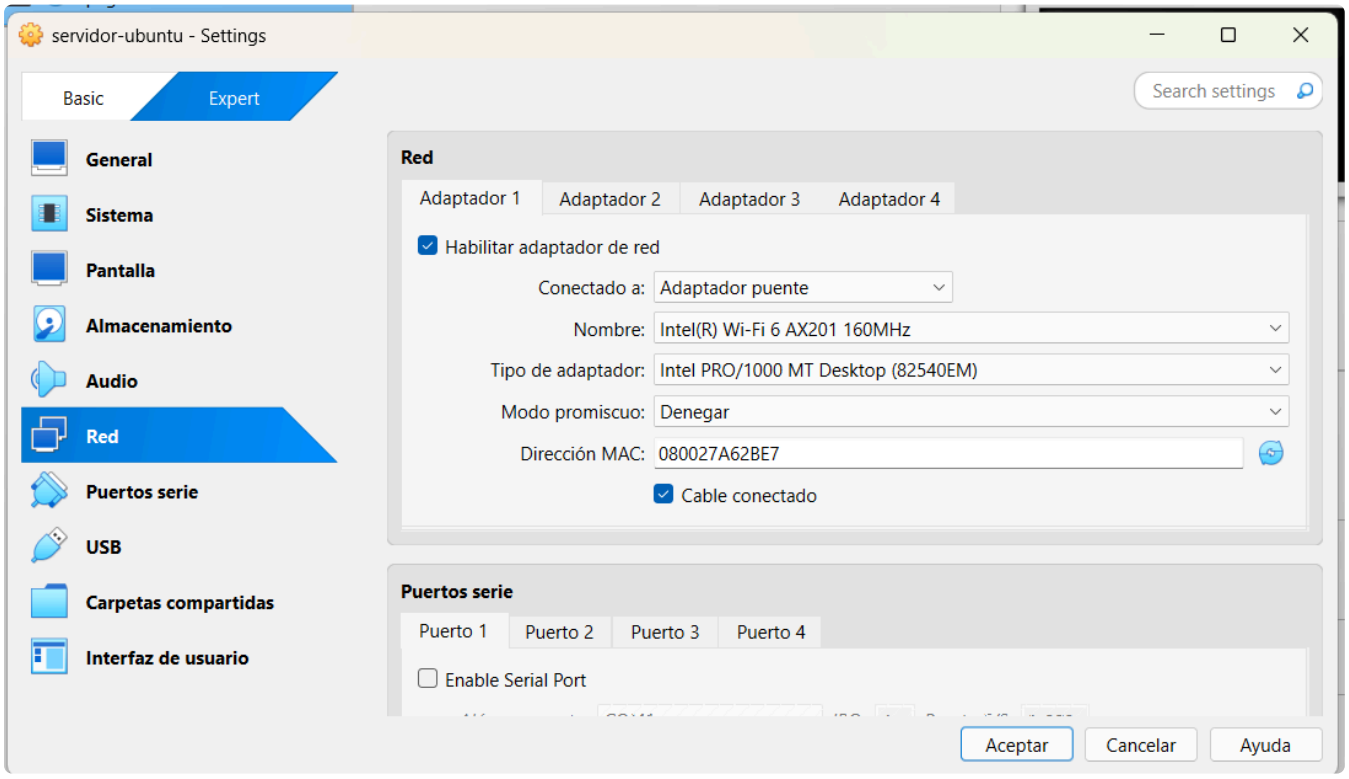


Figura 21. Configuración de red de la máquina virtual. El adaptador 1 está conectado en modo puente sobre la tarjeta Intel Wi-Fi 6 AX201 del equipo anfitrión.

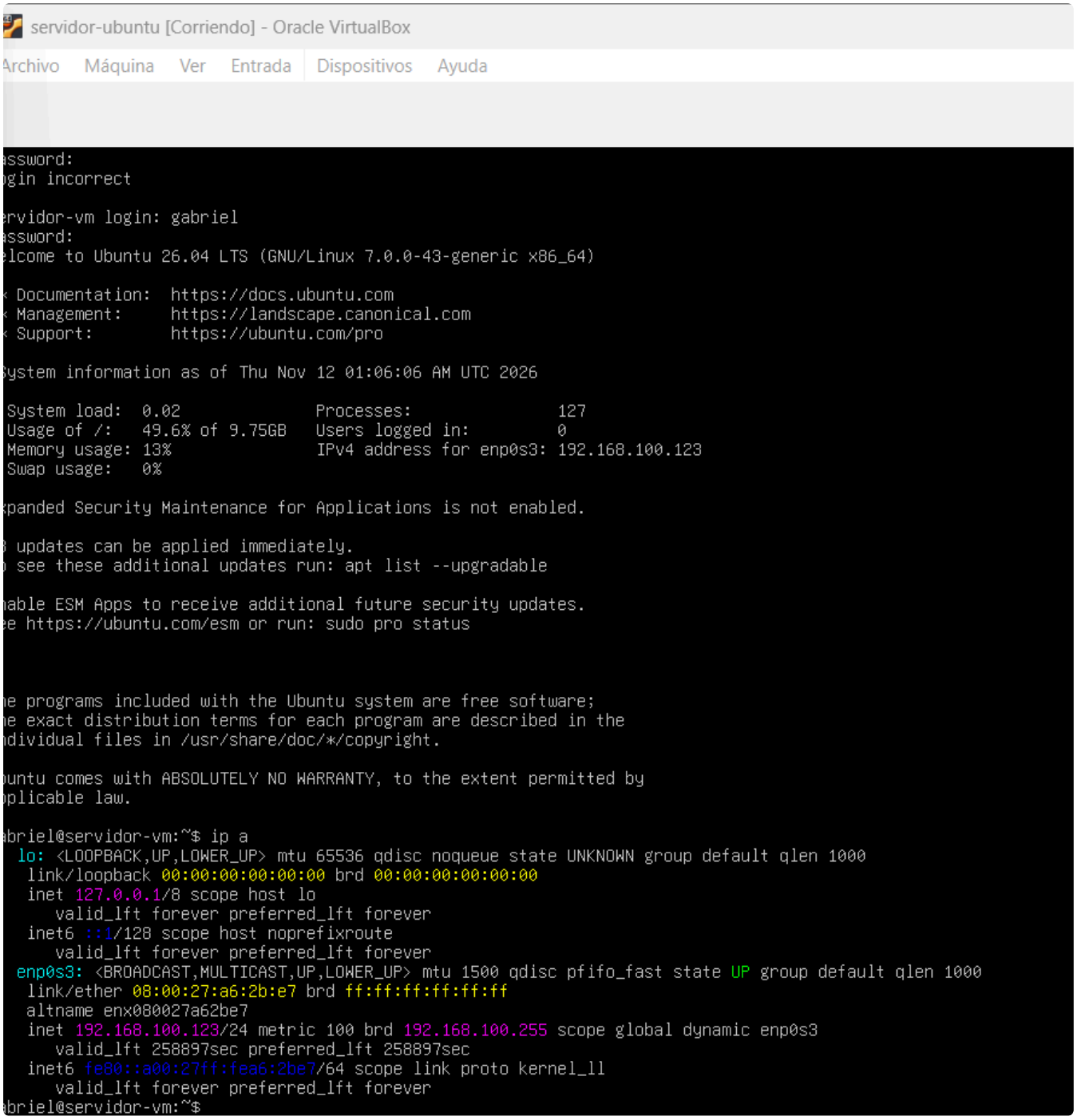


Figura 22. Sesión iniciada en la máquina virtual. La interfaz `enp0s3` tiene la dirección 192.168.100.123, asignada por el router de la red local.

Reparto de roles

Máquina	Rol	Sistema	Prompt visible
Máquina virtual	Servidor	Ubuntu Server 26.04	`gabriel@servidor-vm`
Máquina principal	Cliente	Ubuntu 26.04 sobre WSL2	`gabriel@DESKTOP-P7KUEQK`

Preparación del servidor

En la VM se instaló el JDK y se descargaron los dos archivos del ejemplo directamente desde el repositorio, compilándolos con `javac` en lugar de Gradle. La versión de Gradle disponible en los repositorios de Ubuntu es la 4.4.1, incompatible con las versiones actuales de Java, y el ejemplo no necesita Gradle para funcionar: son dos clases sin dependencias externas.

```
sudo apt update && sudo apt install -y default-jdk
mkdir -p ~/socket && cd ~/socket
wget -q
https://raw.githubusercontent.com/amehlhase316/ser321examples/master/Sockets/
JavaSimpleSock2/src/main/java/SockServer.java
wget -q
https://raw.githubusercontent.com/amehlhase316/ser321examples/master/Sockets/
JavaSimpleSock2/src/main/java/SockClient.java
javac SockServer.java SockClient.java
```

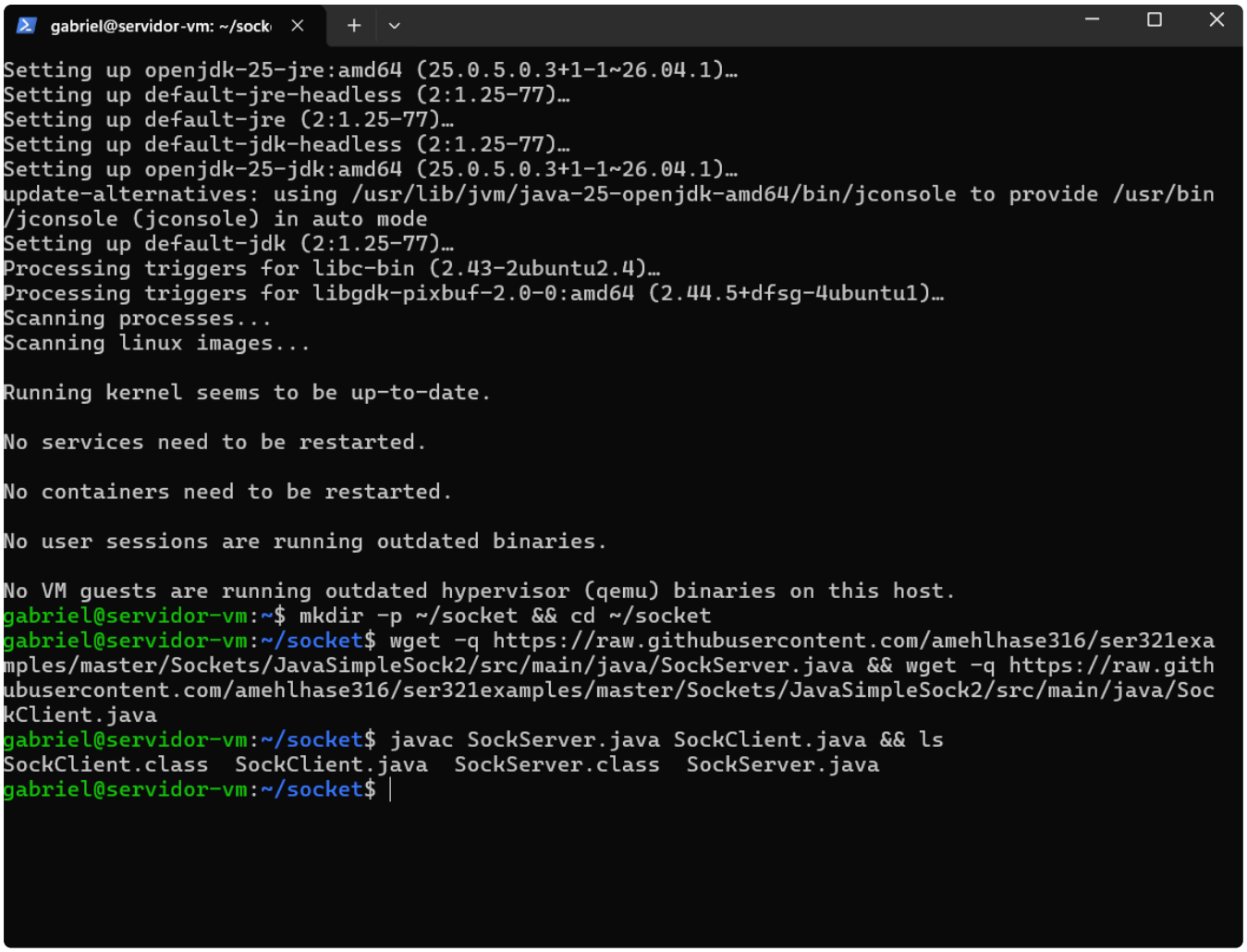


Figura 23. Instalación de OpenJDK 25 en la máquina virtual, descarga de las dos clases del ejemplo y compilación con `javac`.

Ejecución

En la máquina virtual (servidor):

```
cd ~/socket
java SockServer
```

En la máquina principal (cliente), desde la carpeta del ejemplo en el repositorio clonado:

```
gradle SocketClient -Phost=192.168.100.123 -Pmessage="Mensaje desde la
maquina principal" -Pnumber=321 -q --console=plain
```

Resultado

Salida del servidor en la VM:

```
Server ready for 3 connections
Server waiting for a connection
Received the String Mensaje desde la maquina principal
Received the Integer 321
Server waiting for a connection
Received the String Segunda prueba Gabriel 0705081396
Received the Integer 777
Server waiting for a connection
```

Salida del cliente en la máquina principal: `Got it!` en cada una de las dos ejecuciones.

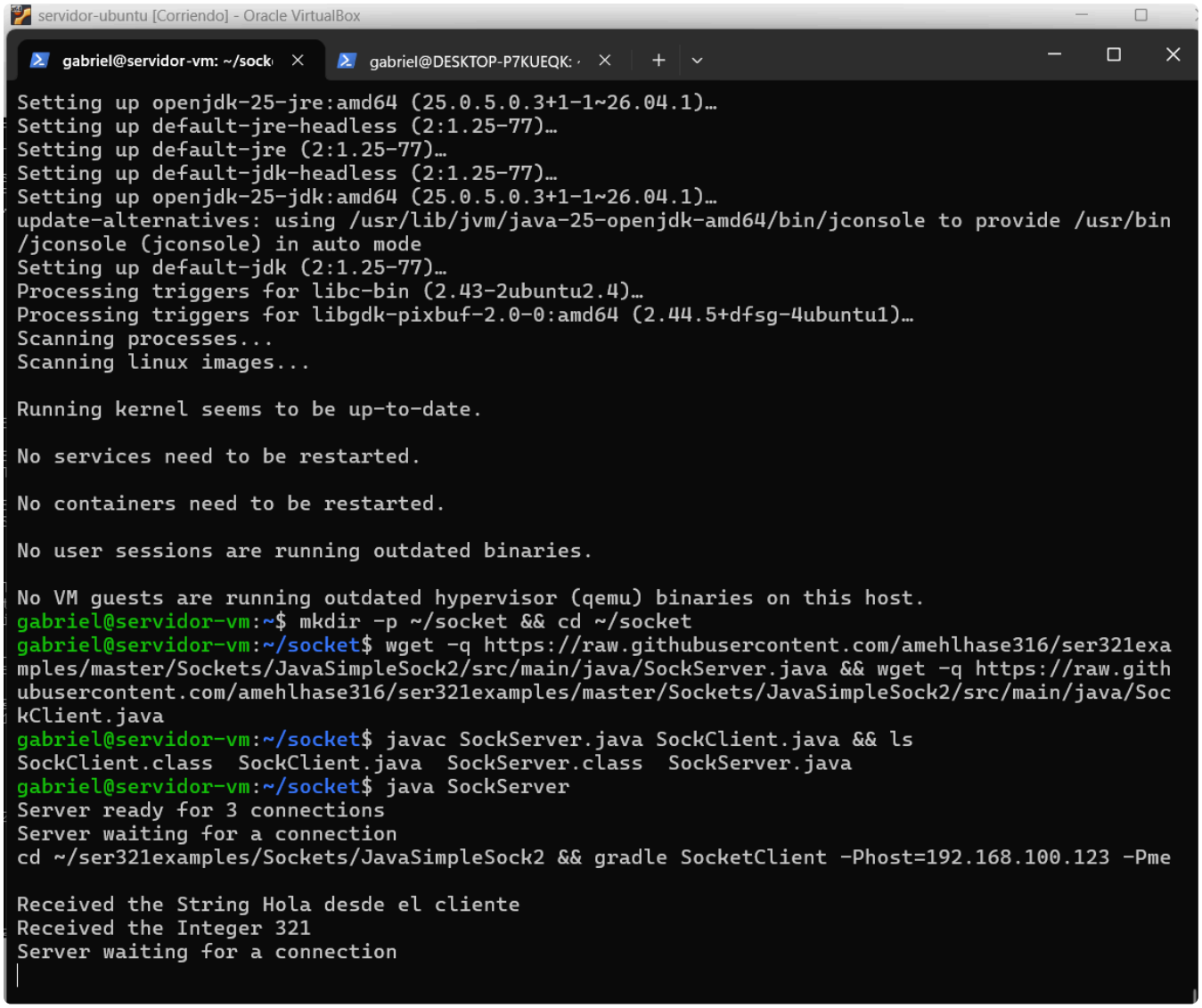


Figura 24. Primera prueba de conexión. El servidor en la máquina virtual recibe la cadena y el entero enviados desde la máquina principal.

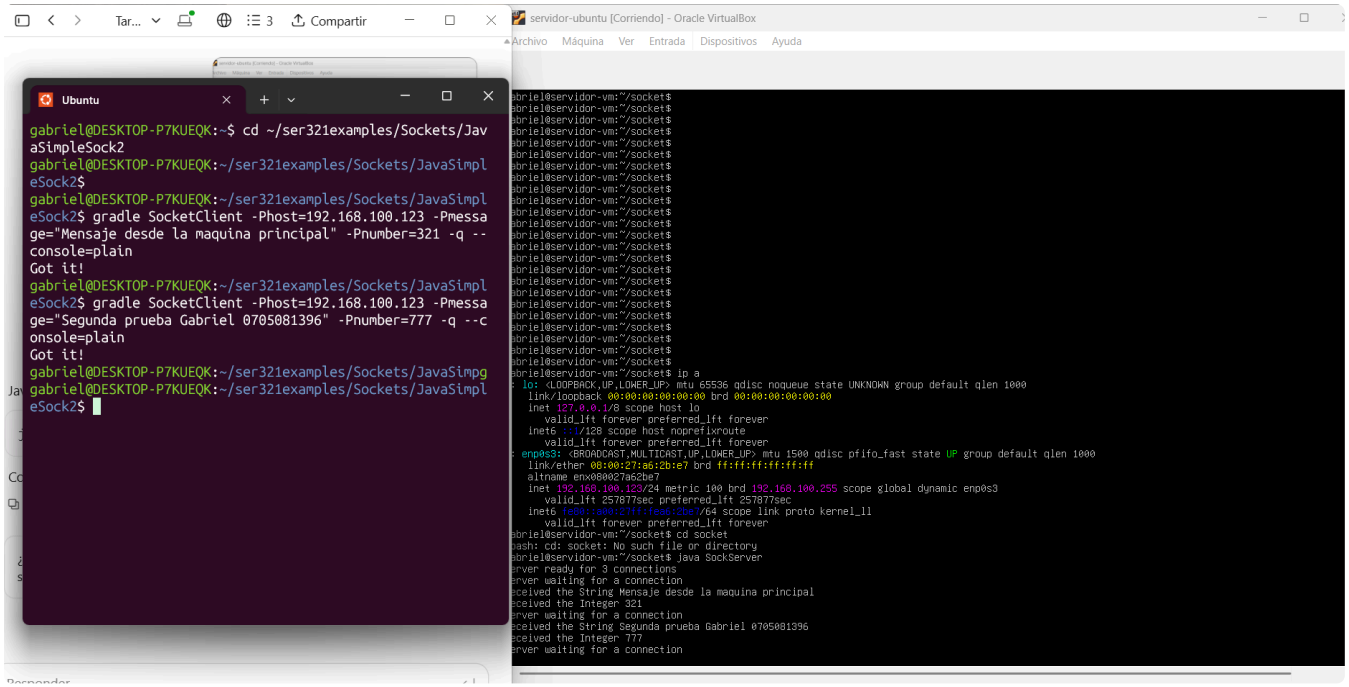


Figura 25. Las dos máquinas en funcionamiento. A la izquierda la terminal del cliente con los dos `Got it!`; a la derecha la ventana de VirtualBox con el servidor. Los prompts `DESKTOP-P7KUEQK` y `servidor-vm` distinguen cada sistema.

```
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$
abriel@servidor-vm:~/socket$ ip a
: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:a6:2b:e7 brd ff:ff:ff:ff:ff:ff
    altname enx080027a62be7
    inet 192.168.100.123/24 metric 100 brd 192.168.100.255 scope global dynamic enp0s3
       valid_lft 257877sec preferred_lft 257877sec
    inet6 fe80::a00:27ff:fea6:2be7/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
abriel@servidor-vm:~/socket$ cd socket
bash: cd: socket: No such file or directory
abriel@servidor-vm:~/socket$ java SockServer
erver ready for 3 connections
erver waiting for a connection
eceived the String Mensaje desde la maquina principal
eceived the Integer 321
erver waiting for a connection
eceived the String Segunda prueba Gabriel 0705081396
eceived the Integer 777
erver waiting for a connection
```

Figura 26. Salida completa del servidor tras las dos conexiones, con el mensaje y el número de cada una.

Cómo funciona el intercambio

`SockServer` abre un `ServerSocket` en el puerto 8888 y se bloquea en `accept()` hasta que llega una conexión. Cuando el cliente conecta, el servidor lee dos objetos serializados por el flujo —primero una cadena, después un entero— y responde con la cadena `"Got it!"`.

El cliente hace lo contrario: abre un `Socket` contra la dirección y el puerto del servidor, escribe los dos objetos con `ObjectOutputStream`, llama a `flush()` para que no se queden en el búfer, y espera la respuesta.

El orden de lectura y escritura es el acuerdo entre las dos partes: el servidor sabe que el primer objeto es una cadena y el segundo un entero porque así lo envía el cliente, no porque el protocolo lo declare. Eso es un protocolo de aplicación —capa 7— construido sobre los sockets TCP de la capa 4.

Una limitación del ejemplo: el servidor atiende exactamente tres conexiones y después termina. El bucle `for (int rep = 0; rep < 3; rep++)` lo fija así, y además es de un solo hilo, de modo que atiende un cliente a la vez.

Video de la demostración

Enlace: *https://drive.google.com/file/d/1xQZvAU2NQmdw-Ad3-U7u3VaUzxyAOZxO/view?usp=sharing*

El video muestra las dos terminales simultáneamente: a la derecha la ventana de VirtualBox con la máquina virtual, donde se ejecuta `ip a` para evidenciar su dirección en la red y luego se arranca el servidor; a la izquierda la terminal de la máquina principal, desde la que se lanza el cliente dos veces con mensajes distintos.